

OCP Maintenance AI Platform — Documentacion Tecnica

Version: 1.0.0

Ultima Actualizacion: Febrero 2026

Plataforma: Aplicacion Web Full-Stack Contenerizada

Cliente: OCP Group (Office Cherifien des Phosphates)

Tabla de Contenidos

1. [Descripcion General](#)
 2. [Arquitectura del Sistema](#)
 3. [Stack Tecnologico](#)
 4. [Estructura del Proyecto](#)
 5. [Backend \(FastAPI\)](#)
 6. [Frontend \(React + Vite\)](#)
 7. [Esquema de Base de Datos](#)
 8. [Autenticacion y Autorizacion](#)
 9. [Referencia de API](#)
 10. [Internacionalizacion \(i18n\)](#)
 11. [Integracion SAP](#)
 12. [Docker y Despliegue](#)
 13. [Modulos Funcionales](#)
 14. [Flujos de Trabajo Clave](#)
 15. [Seguridad](#)
 16. [Guia de Desarrollo Local](#)
 17. [Checklist de Produccion](#)
 18. [Variables de Entorno](#)
-

1. Descripción General

1.1 ¿Qué es este proyecto?

OCP Maintenance AI Platform es un sistema empresarial de gestión de mantenimiento con 4 módulos, potenciado por inteligencia artificial. Fue diseñado para OCP Group, el mayor exportador de fosfatos del mundo, para optimizar las operaciones de mantenimiento en su complejo industrial Jorf Lasfar (JFC1).

1.2 Objetivos Principales

Objetivo	Descripción
Mantenimiento Predictivo	Usar análisis Weibull e IA para predecir fallas de equipos antes de que ocurran
Optimización RCM	Aplicar metodología de Mantenimiento Centrado en Confiabilidad para optimizar estrategias de tareas
Integración SAP	Generar órdenes de trabajo, listas de tareas y paquetes de carga compatibles con SAP
Soporte Multi-Idioma	Interfaz completa en Inglés, Español y Árabe (RTL)
Control de Acceso por Roles	5 roles de usuario con control granular de permisos
Analítica en Tiempo Real	KPIs, puntuaciones de salud, OEE, tableros MTBF/MTTR

1.3 Los 4 Módulos

Modulo 1: JERARQUIA DE ACTIVOS Y CRITICIDAD

Gestión del árbol de equipos, evaluación de matriz de riesgo

Modulo 2: FMEA / FMECA Y ESTRATEGIA RCM

Análisis de modos de falla, lógica de decisión RCM, generación de tareas

Modulo 3: OPERACIONES Y PLANIFICACIÓN

Solicitudes de trabajo, gestión de backlog, programación, paquetes de trabajo

Modulo 4: ANALÍTICA, REPORTES Y MEJORA CONTINUA

KPIs, análisis Weibull, RCA (5W2H), eliminación de defectos, exportación SAP

2. Arquitectura del Sistema

2.1 Diagrama General



2.2 Puertos de los Servicios

Servicio	Puerto	Descripcion
Frontend (Vite)	5173	Servidor de desarrollo React
Backend (FastAPI)	8000	API + Documentacion Swagger en /docs
Nginx (Proxy)	8080	Punto de entrada unificado
Nginx (SSL)	8443	HTTPS (produccion)

2.3 Flujo de Peticiones

```
Navegador -> Nginx:8080
|-- /api/*    -> FastAPI:8000 (API Backend)
|-- /health   -> FastAPI:8000 (Health check)
|-- /docs     -> FastAPI:8000 (Swagger UI)
+-- /*        -> React:5173  (Frontend SPA)
```

3. Stack Tecnológico

3.1 Backend

Tecnologia	Version	Proposito
Python	3.11	Runtime
FastAPI	0.133.1	Framework web
Uvicorn	0.41.0	Servidor ASGI
Gunicorn	25.1.0	Gestor de procesos (produccion)
SQLAlchemy	2.0.47	ORM
Pydantic	2.12.5	Validacion de datos
python-jose	3.3.0	Tokens JWT
bcrypt	4.2.1	Hash de contraseñas
Anthropic SDK	0.83.0	Integracion con Claude AI
openpyxl	3.1.5	Generacion de Excel
httpx	0.28.1	Cliente HTTP

3.2 Frontend

Tecnologia	Version	Proposito
React	19.1.0	Framework UI
Vite	7.3.1	Herramienta de build y servidor dev
Tailwind CSS	4.2.1	Estilos utility-first
React Router DOM	7.13.1	Enrutamiento del lado del cliente
Radix UI	Ultimo	Componentes accesibles headless
Recharts	3.7.0	Visualizacion de datos / graficos
Lucide React	0.575.0	Libreria de iconos
XLSX	0.18.5	Exportacion Excel del lado del cliente
file-saver	2.0.5	Utilidad de descarga de archivos

3.3 Infraestructura

Tecnologia	Proposito
Docker	Contenerizacion
Docker Compose	Orquestacion multi-servicio
Nginx Alpine	Proxy inverso + servicio de archivos estaticos
SQLite	Base de datos de desarrollo
PostgreSQL	Base de datos de produccion

4. Estructura del Proyecto

```
ASSET-MANAGEMENT-SOFTWARE-master/
|
|-- api/                                # === BACKEND (FastAPI) ===
|   |-- main.py                        # Entrada: middleware, routers, health
|   |-- config.py                     # Configuración desde .env
|   |-- schemas.py                   # 50+ modelos Pydantic request/response
|   |-- seed.py                      # Script de semilla de base de datos
|
|   |-- database/
|       |-- connection.py            # Motor SQLAlchemy y sesiones
|       +-- models.py                # 25+ modelos ORM
|
|   |-- routers/                     # Modulos de endpoints API
|       |-- admin.py                 # Semilla, estadísticas, auditoria
|       |-- analytics.py             # Puntuaciones de salud, KPIs, Weibull
|       |-- auth.py                  # Login, registro, JWT
|       |-- backlog.py               # Gestión de backlog
|       |-- capture.py               # Captura de datos en campo
|       |-- criticality.py           # Evaluación de riesgo
|       |-- dashboard.py             # Tablero ejecutivo
|       |-- fmea.py                  # FMEA/FMECA, decisiones RCM
|       |-- hierarchy.py             # CRUD árbol de equipos
|       |-- planner.py               # Recomendaciones del planificador IA
|       |-- rca.py                   # Analisis de Causa Raiz
|       |-- reliability.py           # Repuestos, paradas, MOCs
|       |-- reporting.py             # Reportes, exportaciones, notificaciones
|       |-- sap.py                   # Integración SAP
|       |-- scheduling.py            # Programación, GANTT
|       |-- tasks.py                 # CRUD tareas de mantenimiento
|       |-- work_packages.py         # Agrupación de paquetes de trabajo
|       +-- work_requests.py         # Validación de solicitudes de trabajo
|
|   |-- services/                    # Capa de lógica de negocio (23 servicios)
|       |-- agent_service.py         # Configuración del agente IA
|       |-- analytics_service.py     # Cálculos de KPI
|       |-- audit_service.py         # Registro de auditoria
|       |-- auth_service.py          # JWT, hash de contraseñas
|       |-- backlog_service.py       # Optimización de backlog
|       |-- criticality_service.py   # Motor de matriz de riesgo
|       |-- fmea_service.py          # Lógica FMEA + RCM
|       |-- hierarchy_service.py     # Jerarquía de equipos
|       |-- planner_service.py       # Algoritmos de planificación
|       |-- rca_service.py           # Analisis 5W2H
|       |-- reliability_service.py   # Predicción de fallas
|       |-- reporting_service.py     # Generación de reportes
|       |-- sap_service.py           # Procesamiento de datos SAP
|       |-- scheduling_service.py    # Optimización de programación
|       |-- work_package_service.py  # Lógica de agrupación WP
|       |-- work_request_service.py  # Procesamiento de WR
|       +-- ...                      # Servicios adicionales
|
|   +-- dependencies/
|       +-- auth.py                  # Inyección de dependencias JWT
|
|-- frontend/                          # === FRONTEND (React SPA) ===
|   |-- package.json                  # Dependencias npm
```

```

|-- vite.config.js      # Configuración Vite + proxy
|-- index.html          # Entrada SPA (favicon, meta)
|-- Dockerfile          # Build multi-etapa
|
+-- src/
|   |-- main.jsx        # Configuración de rutas (23 rutas)
|   |-- App.jsx         # Layout raíz (Sidebar + Header)
|   |-- api.js          # Cliente API (50+ endpoints)
|
|   |-- pages/          # 23 componentes de página
|   |   |-- Login.jsx   # Autenticación
|   |   |-- Dashboard.jsx # Vista general de KPIs
|   |   |-- Hierarchy.jsx # Árbol de equipos
|   |   |-- Criticality.jsx # Evaluación de riesgo
|   |   |-- FMEA.jsx    # Análisis de modos de falla
|   |   |-- FMECA.jsx   # FMEA con criticidad
|   |   |-- Strategy.jsx # Selección de estrategia RCM
|   |   |-- WorkRequests.jsx # Gestión de solicitudes
|   |   |-- WorkPackages.jsx # Agrupación de tareas + exportación SAP
|   |   |-- Backlog.jsx  # Backlog de mantenimiento
|   |   |-- Scheduling.jsx # Programación
|   |   |-- Planning.jsx # Planificación a largo plazo
|   |   |-- Planner.jsx  # Chat asistente IA
|   |   |-- FieldCapture.jsx # Captura de datos móvil
|   |   |-- Analytics.jsx # Tableros de KPI
|   |   |-- ExecutiveDashboard.jsx # Resumen ejecutivo
|   |   |-- Reports.jsx  # Generación de reportes + exportación
|   |   |-- Reliability.jsx # Repuestos, paradas
|   |   |-- RCA.jsx      # Análisis de Causa Raíz
|   |   |-- DefectElimination.jsx # Seguimiento de defectos
|   |   |-- SAPReview.jsx # Revisión de cargas SAP
|   |   |-- Admin.jsx    # Gestión de usuarios + configuración
|   |   +-- Profile.jsx  # Perfil del usuario
|
|   |-- components/     # Componentes UI reutilizables
|   |   |-- Sidebar.jsx  # Menu de navegación
|   |   |-- Header.jsx   # Barra superior + selector de planta
|   |   |-- ProtectedRoute.jsx # Guardia de acceso por roles
|   |   |-- ErrorBoundary.jsx # Manejo de errores
|   |   |-- Toast.jsx    # Sistema de notificaciones
|   |   |-- ConfirmDialog.jsx # Diálogos de confirmación
|   |   |-- Shared.jsx   # StatusBadge, LoadingSpinner
|   |   +-- ui/         # 15+ primitivas Radix UI
|
|   |-- contexts/       # Estado global
|   |   |-- AuthContext.jsx # Estado de autenticación
|   |   +-- LanguageContext.jsx # Selección de idioma i18n
|
|   |-- i18n/           # Traducciones
|   |   |-- en.js       # Inglés
|   |   |-- es.js       # Español
|   |   +-- ar.js       # Árabe (RTL)
|
|   |-- data/
|   |   +-- mockData.js  # Datos mock de desarrollo
|
|   |-- utils/          # Funciones utilitarias
|   |   +-- exportFile.js # Helper de descarga Excel/CSV
|
+-- styles/

```

```
|
|         +-- index.css                # CSS global + modo oscuro
|
|-- sap_mock/                          # === SERVICIO MOCK SAP ===
|   |-- generate_mock_data.py          # Generador de datos mock
|   +-- data/                          # Datasets JSON
|       |-- work_orders.json           # Ordenes de trabajo historicas
|       |-- functional_locations.json  # Ubicaciones de equipos
|       |-- materials_bom.json         # Lista de materiales
|       |-- equipment_master.json      # Catalogo de equipos
|       +-- maintenance_plans.json     # Planes predefinidos
|
|-- Dockerfile                         # Contenedor del backend
|-- docker-compose.yml                 # Orquestacion de 3 servicios
|-- nginx.conf                         # Configuracion del proxy inverso
|-- requirements.txt                   # Dependencias Python (27 paquetes)
|-- .env.example                       # Plantilla de variables de entorno
+-- ocp_maintenance.db                 # Base de datos SQLite (dev)
```

5. Backend (FastAPI)

5.1 Inicializacion de la Aplicacion

La aplicacion se crea en `api/main.py` mediante la fabrica `create_app()` :

1. **Instancia FastAPI** con Swagger UI en `/docs`
2. **Middleware CORS** — permite origenes del frontend desde `.env`
3. **Middleware de API Key** — aplica `X-API-Key` en mutaciones cuando esta configurado
4. **18 routers** registrados bajo el prefijo `/api/v1`
5. **Tablas de base de datos** creadas al inicio mediante el manejador lifespan
6. **Endpoint de salud** en `/health` con verificacion de conectividad DB

5.2 Routers (18 Modulos)

Router	Prefijo	Proposito
auth	/auth	Login, registro, tokens JWT, gestion de usuarios
hierarchy	/hierarchy	CRUD de plantas y nodos de equipos
criticality	/criticality	Evaluacion de matriz de riesgo y aprobacion
fmea	/fmea	Funciones, fallas, modos de falla, RCM, FMECA
tasks	/tasks	CRUD de tareas de mantenimiento
work_packages	/work-packages	Agrupacion de tareas, aprobacion, instrucciones
sap	/sap	Generacion de cargas SAP, acceso a datos mock
analytics	/analytics	Puntuaciones de salud, KPIs, analisis Weibull
work_requests	/work-requests	Validacion de WR, duplicados, OCR
capture	/capture	Captura de datos en campo
planner	/planner	Recomendaciones del planificador IA
backlog	/backlog	Gestion y optimizacion de backlog
scheduling	/scheduling	Programacion, GANTT
reliability	/reliability	Repuestos, paradas, MOCs
reporting	/reporting	Reportes, notificaciones, exportacion
dashboard	/dashboard	KPIs ejecutivos y alertas
rca	/rca	Analisis de Causa Raiz (5W2H)
admin	/admin	Semilla, estadisticas, auditoria, feedback

5.3 Capa de Servicios

Cada router delega la logica de negocio a un servicio correspondiente en `api/services/`. Servicios clave:

- `auth_service.py` — Hash de contraseñas (bcrypt), creacion/validacion JWT, CRUD de usuarios
- `fmea_service.py` — Logica FMEA/FMECA, arboles de decision RCM, calculo RPN
- `analytics_service.py` — Ajuste Weibull, computo de puntuacion de salud, agregacion KPI
- `criticality_service.py` — Evaluacion de matriz de riesgo, clases de riesgo sugeridas por IA
- `work_package_service.py` — Algoritmos de agrupacion de tareas, generacion de cargas SAP

- `reporting_service.py` — Generación de reportes semanales/mensuales, exportación Excel
- `rca_service.py` — Framework de análisis de causa raíz 5W2H
- `capture_service.py` — Procesamiento de capturas de campo con detección de modos de falla

5.4 Configuración

`api/config.py` carga configuraciones desde variables de entorno:

```

DATABASE_URL      # Cadena de conexión SQLite o PostgreSQL
SAP MOCK DIR      # Ruta al directorio de datos mock SAP
JWT_SECRET_KEY    # Secreto para firmar tokens JWT
JWT_ALGORITHM     # HS256 (por defecto)
CORS_ORIGINS      # Orígenes permitidos separados por coma
API_V1_PREFIX     # /api/v1 (por defecto)
LOG_LEVEL         # INFO, DEBUG, WARNING, etc.

```

6. Frontend (React + Vite)

6.1 Entrada de la Aplicación

`main.jsx` configura React Router con 23 rutas lazy-loaded envueltas en guardias basadas en roles:

```

/login          -> Login.jsx          (publico)
/               -> Dashboard.jsx       (todos los roles)
/hierarchy      -> Hierarchy.jsx       (todos los roles)
/criticality    -> Criticality.jsx      (todos los roles)
/fmea           -> FMEA.jsx            (planner, engineer, admin)
/fmeca          -> FMECA.jsx           (planner, engineer, admin)
/strategy       -> Strategy.jsx        (planner, engineer, admin)
/work-requests  -> WorkRequests.jsx     (todos los roles)
/work-packages  -> WorkPackages.jsx    (planner, admin)
/backlog        -> Backlog.jsx         (planner, admin)
/scheduling     -> Scheduling.jsx      (planner, admin)
/planner        -> Planner.jsx         (planner, admin)
/planning       -> Planning.jsx        (planner, admin)
/field-capture  -> FieldCapture.jsx     (todos los roles)
/analytics      -> Analytics.jsx       (manager, admin)
/executive      -> ExecutiveDashboard (manager, admin)
/reports        -> Reports.jsx         (manager, admin)
/reliability    -> Reliability.jsx     (engineer, admin)
/rca            -> RCA.jsx             (engineer, admin)
/defect-elimination -> DefectElimination (engineer, admin)
/sap-review     -> SAPReview.jsx       (manager, admin)
/admin          -> Admin.jsx          (solo admin)
/profile        -> Profile.jsx         (todos los roles)

```

6.2 Sistema de Layout

`App.jsx` proporciona el layout principal:

- **Sidebar** — Navegacion verde (#1B5E20) con logo OCP, items de menu filtrados por rol, colapsable
- **Header** — Selector de planta, selector de idioma (EN/ES/AR), modo oscuro, menu de usuario
- **Area de Contenido** — Renderiza la pagina activa via `<Outlet />`

6.3 Gestion de Estado

Contexto	Archivo	Proposito
<code>AuthContext</code>	<code>contexts/AuthContext.jsx</code>	Sesion de usuario, tokens JWT, login/logout, verificacion de roles
<code>LanguageContext</code>	<code>contexts/LanguageContext.jsx</code>	Idioma activo, funcion traductora <code>t()</code> , deteccion RTL

6.4 Cliente API

`api.js` proporciona 50+ funciones que envuelven llamadas HTTP al backend:

- URL base: `/api/v1` (proxy Vite en dev, Nginx en prod)
- Header automatico `Authorization: Bearer {token}`
- Refresco automatico de token ante respuestas 401
- Manejo centralizado de errores

Ejemplo:

```
export const listWorkRequests = (params) => get('/work-requests', params);
export const validateWorkRequest = (id, d) => put(`/work-requests/${id}/validate`, d);
export const calculateHealthScore = (d) => post('/analytics/health-score', d);
```

6.5 Libreria de Componentes UI

Construida sobre **Radix UI** (headless, accesible) + **Tailwind CSS** (utility-first):

Componente	Descripcion
<code>Sidebar.jsx</code>	Navegacion principal con filtrado por roles
<code>Header.jsx</code>	Barra superior con selectores de planta/idioma/tema
<code>ProtectedRoute.jsx</code>	Guardia de ruta que verifica roles de usuario
<code>Toast.jsx</code>	Toasts de notificacion exito/error/info
<code>ConfirmDialog.jsx</code>	Dialogos modales de confirmacion
<code>Shared.jsx</code>	<code>StatusBadge</code> , <code>LoadingSpinner</code> , <code>DataTable</code>
<code>ui/*.jsx</code>	15+ primitivas Radix (Button, Dialog, Select, Tabs, etc.)

6.6 Tematizacion

- **Modo claro/oscuro** via propiedades CSS personalizadas (`--background` , `--foreground` , etc.)
- **Color de marca OCP:** `#1B5E20` (verde oscuro) como primario
- **Sidebar:** Fondo verde solido `bg-[#1B5E20]`
- **Tarjetas:** `bg-card border-border` adaptativo al tema
- **Badges de estado:** Codificados por color (verde=aprobado, ambar=borrador, azul=revision, rojo=critico)

7. Esquema de Base de Datos

7.1 Modelos Principales (25+ Tablas)

```
Usuarios y Autenticacion
|-- UserModel          # Usuarios con roles (admin, manager, planner, tecnico)

Jerarquia de Activos
|-- PlantModel         # Instalaciones de manufactura
|-- HierarchyNodeModel # Arbol de equipos (6 niveles de profundidad)

Criticidad
|-- CriticalityAssessmentModel # Evaluaciones de matriz de riesgo con flujo de aprobacion

FMEA / FMECA
|-- FunctionModel      # Funciones del equipo
|-- FunctionalFailureModel # Fallas funcionales (TOTAL/PARCIAL)
|-- FailureModeModel   # Modos de falla con mecanismo/causa
|-- MaintenanceTaskModel # Tareas derivadas de RCM (CBM/TBM/FTM/RTF)

Gestion de Trabajo
|-- WorkPackageModel   # Tareas de mantenimiento agrupadas
|-- WorkOrderModel     # Ordenes de trabajo historicas SAP

Analitica
|-- HealthScoreModel   # Salud del activo multidimensional
|-- KPIMetricsModel    # MTBF, MTTR, disponibilidad, OEE
|-- FailurePredictionModel # Predicciones basadas en Weibull

Integracion SAP
|-- SAPUploadPackageModel # Paquetes de exportacion SAP

Mejora Continua
|-- CAPAItemModel      # Acciones correctivas/preventivas
```

7.2 Niveles de Jerarquia de Equipos

```
PLANTA (ej., OCP-JFC1)
+-- AREA (ej., Molienda)
+-- SISTEMA (ej., Circuito Molino SAG)
+-- EQUIPO (ej., Molino SAG #1)
+-- SUB_ENSAMBLE (ej., Conjunto de Accionamiento)
+-- ITEM_MANTENIBLE (ej., Rodamiento Principal)
```

8. Autenticacion y Autorizacion

8.1 Flujo de Tokens JWT

1. POST /auth/login { username, password }
-> Retorna: { access_token (4h), refresh_token (7d), user }
2. Todas las peticiones API incluyen:
Authorization: Bearer <access_token>
3. Ante 401 (token expirado):
POST /auth/refresh { refresh_token }
-> Retorna: nuevo access_token + refresh_token
4. Si el refresco falla:
-> Redireccion a /login

8.2 Roles de Usuario y Permisos

Rol	Acceso
admin	Acceso total al sistema + gestion de usuarios
manager	Supervision de operaciones, analitica, reportes, revision SAP
planner	FMEA, paquetes de trabajo, backlog, programacion, planificador IA
tecnico	Dashboard, captura en campo, solicitudes de trabajo, jerarquia
engineer	Confiabilidad, RCA, eliminacion de defectos, FMEA

8.3 Usuarios Demo por Defecto

Usuario	Contraseña	Rol
admin	admin123	admin
manager	manager123	manager
planner	planner123	planner
tecnico	tecnico123	tecnico

8.4 Seguridad de Contraseñas

- Hash con **bcrypt** (rondas de sal)
- Nunca almacenadas en texto plano
- Validacion minima en el registro

- Cambio de contraseña requiere verificación de la contraseña anterior

9. Referencia de API

9.1 URL Base

```
Desarrollo: http://localhost:8000/api/v1
Produccion: https://tu-dominio.com/api/v1
Swagger UI: http://localhost:8000/docs
```

9.2 Resumen de Endpoints

Autenticación (/auth)

Metodo	Endpoint	Descripcion
POST	/auth/login	Iniciar sesión con credenciales
POST	/auth/refresh	Refrescar tokens JWT
POST	/auth/register	Crear nuevo usuario (admin)
GET	/auth/me	Perfil del usuario actual
PUT	/auth/me	Actualizar perfil
PUT	/auth/change-password	Cambiar contraseña
GET	/auth/users	Listar todos los usuarios (admin)
PUT	/auth/users/{id}/role	Actualizar rol de usuario
PUT	/auth/users/{id}/activate	Activar usuario
PUT	/auth/users/{id}/deactivate	Desactivar usuario

Jerarquía (/hierarchy)

Metodo	Endpoint	Descripcion
GET	<code>/hierarchy/plants</code>	Listar plantas
POST	<code>/hierarchy/plants</code>	Crear planta
GET	<code>/hierarchy/nodes</code>	Listar nodos de equipos
POST	<code>/hierarchy/nodes</code>	Crear nodo
GET	<code>/hierarchy/nodes/{id}</code>	Obtener detalles del nodo
GET	<code>/hierarchy/nodes/{id}/tree</code>	Obtener subarbol
POST	<code>/hierarchy/build-from-vendor</code>	Importar desde datos del proveedor
GET	<code>/hierarchy/stats</code>	Estadísticas de jerarquia

FMEA (`/fmea`)

Metodo	Endpoint	Descripcion
POST	<code>/fmea/functions</code>	Crear funcion del equipo
GET	<code>/fmea/functions</code>	Listar funciones
POST	<code>/fmea/functional-failures</code>	Crear falla funcional
POST	<code>/fmea/failure-modes</code>	Crear modo de falla
GET	<code>/fmea/failure-modes</code>	Listar modos de falla
GET	<code>/fmea/validate-fm</code>	Validar combinacion FM
GET	<code>/fmea/fm-combinations</code>	Pares mecanismo/causa validos
POST	<code>/fmea/rcm-decide</code>	Decision de estrategia RCM
POST	<code>/fmea/fmea/worksheets</code>	Crear hoja de trabajo FMECA
POST	<code>/fmea/fmea/rpn</code>	Calcular RPN

Analitica (`/analytics`)

Metodo	Endpoint	Descripcion
POST	<code>/analytics/health-score</code>	Calcular salud del activo
POST	<code>/analytics/kpis</code>	Calcular metricas KPI
POST	<code>/analytics/weibull-fit</code>	Ajustar distribucion Weibull
POST	<code>/analytics/weibull-predict</code>	Predecir proxima falla
GET	<code>/analytics/variance-alerts</code>	Anomalias estadisticas
GET	<code>/analytics/asset-health</code>	Resumen de salud

Reportes (`/reporting`)

Metodo	Endpoint	Descripcion
POST	<code>/reporting/reports/weekly</code>	Generar reporte semanal
POST	<code>/reporting/reports/monthly</code>	Generar reporte mensual
POST	<code>/reporting/reports/quarterly</code>	Generar reporte trimestral
GET	<code>/reporting/reports</code>	Listar reportes
POST	<code>/reporting/export</code>	Exportar datos (Excel/CSV/SAP)
GET	<code>/reporting/notifications</code>	Listar notificaciones
POST	<code>/reporting/cross-module/analyze</code>	Analisis cross-modulo

10. Internacionalizacion (i18n)

10.1 Idiomas Soportados

Idioma	Codigo	Direccion	Archivo
Ingles	<code>en</code>	LTR	<code>frontend/src/i18n/en.js</code>
Espanol	<code>es</code>	LTR	<code>frontend/src/i18n/es.js</code>
Arabe	<code>ar</code>	RTL	<code>frontend/src/i18n/ar.js</code>

10.2 Implementacion

El sistema i18n usa React Context (`LanguageContext.jsx`):

```
// Acceder a traducciones en cualquier componente:
const { t, lang, setLang } = useLanguage();

// Uso:
t('nav.dashboard') // -> "Panel Principal"
t('common.noResults', { query: 'bomba' }) // -> "Sin resultados para 'bomba'"
t('planner.suggestions') // -> ["Sugerencia 1", "Sugerencia 2", ...]
```

10.3 Estructura de Traducciones

```
{
  common: { // Etiquetas UI compartidas, botones, estados
  auth: { // Login, roles, credenciales
  nav: { // Items del menu de navegacion
  hierarchy: { // Terminos de jerarquia de equipos
  criticality: { // Terminos de evaluacion de riesgo
  fmea: { // Terminos de analisis de modos de falla
  tasks: { // Gestion de tareas
  workPackages: { // Operaciones de paquetes de trabajo
  analytics: { // Etiquetas y metricas de KPI
  reports: { // Tipos de reporte y exportacion
  planner: { // Interfaz del planificador IA + respuestas
  strategy: { // Etiquetas de estrategia RCM
  admin: { // Panel de administracion
  // ... mas modulos
}
```

10.4 Soporte RTL

El layout en arabe aplica automaticamente `dir="rtl"` al elemento raiz, invirtiendo flexbox, margenes, padding y alineacion de texto.

11. Integracion SAP

11.1 Vision General

La plataforma genera paquetes de datos compatibles con SAP para carga en el modulo SAP PM (Plant Maintenance). Durante el desarrollo, un servicio mock SAP proporciona datos de prueba realistas.

11.2 Datos Mock (`sap_mock/data/`)

Archivo	Transaccion SAP	Contenido
<code>work_orders.json</code>	IW31/IW32	Ordenes PM historicas
<code>functional_locations.json</code>	IL01	Ubicaciones tecnicas
<code>materials_bom.json</code>	CS01	Lista de materiales
<code>equipment_master.json</code>	IE01	Registros maestros de equipos
<code>maintenance_plans.json</code>	IP10	Planes de mantenimiento predefinidos

11.3 Flujo de Carga SAP

1. Crear Paquete de Trabajo (agrupar tareas)
2. Aprobar Paquete de Trabajo
3. Generar Paquete de Carga SAP
4. Revisar en pagina de Revision SAP
5. Aprobar para transferencia SAP
6. Exportar en formato compatible SAP

11.4 Mapeo de Campos SAP

Campo OCP	-> Campo SAP
<code>work_order_id</code>	-> AUFNR (Numero de Orden)
<code>equipment_tag</code>	-> EQUNR (Numero de Equipo)
<code>task_type (PM01-03)</code>	-> AUART (Tipo de Orden)
<code>priority</code>	-> PRIOK (Prioridad)
<code>description</code>	-> KTEXT (Texto Breve)
<code>planned_hours</code>	-> ARBEI (Trabajo Planificado)

12. Docker y Despliegue

12.1 Servicios (docker-compose.yml)

```
services:
  ocp-backend:      # FastAPI + Gunicorn (4 workers)
    puertos: 8000
    volúmenes: ocp_db_data, ocp_sap_data
    healthcheck: HTTP GET /health

  ocp-frontend:     # Build React servido por Nginx
    puertos: 5173
    depende_de: ocp-backend (healthy)

  ocp-nginx:        # Proxy inverso
    puertos: 8080 (HTTP), 8443 (HTTPS)
    config: nginx.conf
```

12.2 Proceso de Build

Backend (Dockerfile):

```
Etapas 1: Python 3.11-slim -> Instalar dependencias
Etapas 2: Copiar paquetes -> Crear usuario no-root -> Ejecutar Gunicorn
```

Frontend (frontend/Dockerfile):

```
Etapas 1: Node 20-alpine -> npm ci -> npm run build
Etapas 2: Nginx Alpine -> Copiar dist/ -> Enrutamiento fallback SPA
```

12.3 Comandos Docker

```
# Construir e iniciar todos los servicios
docker-compose up -d --build

# Ver logs
docker-compose logs -f ocp-backend

# Detener todos los servicios
docker-compose down

# Reconstruir un solo servicio
docker-compose up -d --build ocp-frontend
```

12.4 Configuración Nginx

- **Rate limiting:** 30 req/s general, 2 req/s endpoints admin
- **Headers de seguridad:** X-Frame-Options, X-Content-Type-Options, X-XSS-Protection

- **Proxy pass:** `/api` y `/health` al backend, todo lo demas al frontend
 - **SSL listo:** Configuración HTTPS comentada disponible para producción
-

13. Modulos Funcionales

Modulo 1: Jerarquia de Activos y Criticidad

Paginas: Hierarchy, Criticality

- Construir arbol de equipos desde datos maestros SAP o entrada manual
- Jerarquia de 6 niveles: Planta -> Area -> Sistema -> Equipo -> Sub-Ensamble -> Item Mantenible
- Evaluación de matriz de riesgo con clases de riesgo sugeridas por IA
- Criterios: Seguridad, Medio Ambiente, Producción, Calidad, Costo de Mantenimiento
- Salida: Clasificación de riesgo (ALTO / MEDIO / BAJO)

Modulo 2: FMEA / FMECA y Estrategia RCM

Paginas: FMEA, FMECA, Strategy

- Definir funciones del equipo y fallas funcionales
- Crear modos de falla con mecanismos y causas estandarizados
- Cálculo RPN (Número de Prioridad de Riesgo): Severidad x Ocurrencia x Detección
- Lógica de Decisión RCM:
- **CBM** — Mantenimiento Basado en Condición (predictivo)
- **TBM** — Mantenimiento Basado en Tiempo (preventivo)
- **FTM** — Tarea de Búsqueda de Falla (fallas ocultas)
- **RTF** — Ejecutar Hasta la Falla (sin acción, aceptar riesgo)
- Generar tareas de mantenimiento desde decisiones RCM

Modulo 3: Operaciones y Planificación

Paginas: WorkRequests, WorkPackages, Backlog, Scheduling, Planning, Planner, FieldCapture

- Técnicos de campo capturan datos via formularios móviles
- Solicitudes de trabajo con clasificación de prioridad asistida por IA
- Detección de duplicados para solicitudes similares
- Gestión de backlog con análisis de envejecimiento
- Agrupación de tareas en paquetes de trabajo por área/ventana de parada
- Programación con visualización GANTT
- Interfaz de chat del Planificador IA con recomendaciones contextuales

Modulo 4: Analitica, Reportes y Mejora Continua

Paginas: Analytics, ExecutiveDashboard, Reports, Reliability, RCA, DefectElimination, SAPReview

- **KPIs:** MTBF, MTTR, Disponibilidad, OEE, Adherencia al Programa
 - **Analisis Weibull:** Ajustar distribuciones de falla, predecir proxima falla
 - **Puntuaciones de Salud:** Evaluacion multidimensional de salud del activo
 - **Reportes:** Semanales, mensuales, trimestrales — exportables a Excel/CSV
 - **RCA:** Framework 5W2H (Quien, Que, Cuando, Donde, Por que, Como, Cuanto)
 - **Eliminacion de Defectos:** Seguimiento de fallas recurrentes y acciones correctivas
 - **Revision SAP:** Auditar paquetes de carga generados antes de la transferencia
-

14. Flujos de Trabajo Clave

14.1 Evaluacion de Criticidad

Seleccionar Equipo -> Llenar Criterios de Riesgo -> IA Sugiere Clase de Riesgo
-> Usuario Aprueba/Modifica -> Guardar Evaluacion -> Alimentar FMEA

14.2 FMEA a Paquete de Trabajo

Definir Funciones -> Identificar Fallas Funcionales -> Crear Modos de Falla
-> Aplicar Decision RCM -> Generar Tareas -> Agrupar en Paquetes de Trabajo
-> Aprobar -> Exportar a SAP

14.3 Ciclo de Vida de Solicitud de Trabajo

Captura en Campo -> Crear Solicitud de Trabajo -> Verificar Duplicados
-> Clasificacion de Prioridad IA -> Validar -> Agregar al Backlog
-> Programar -> Ejecutar -> Cerrar (soporte OCR)

14.4 Generacion y Exportacion de Reportes

Seleccionar Tipo de Reporte -> Configurar Parametros -> Generar (API o Mock)
-> Previsualizar Resultados -> Exportar Excel/CSV -> Descargar

15. Seguridad

Característica	Implementación
Autenticación	Tokens JWT (HS256), acceso 4 horas / refresco 7 días
Almacenamiento de Contraseñas	Hash bcrypt con sal
CORS	Lista blanca de orígenes desde entorno
API Key	Aplicación opcional de <code>X-API-Key</code> en mutaciones
Rate Limiting	Nginx: 30 req/s general, 2 req/s admin
Headers de Seguridad	X-Frame-Options: DENY, X-Content-Type-Options: nosniff
HTTPS	Nginx SSL/TLS 1.2+ listo (configuración producción provista)
Acceso por Roles	Guardias de ruta frontend + middleware backend
Contenedor No-Root	Backend ejecuta como <code>appuser</code> (UID 1000)
Protección SQL Injection	Consultas parametrizadas ORM SQLAlchemy
Protección CSRF	Token Bearer JWT (no basado en cookies)

16. Guía de Desarrollo Local

16.1 Prerequisitos

- Python 3.11+
- Node.js 20+
- npm 9+
- Git

16.2 Configuración del Backend

```
# Clonar y navegar
cd ASSET-MANAGEMENT-SOFTWARE-master

# Crear entorno virtual
python -m venv .venv
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate      # Windows

# Instalar dependencias
pip install -r requirements.txt

# Crear .env (copiar desde ejemplo)
cp .env.example .env

# Iniciar servidor backend
PYTHONPATH=. python -m uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
```

16.3 Configuración del Frontend

```
cd frontend

# Instalar dependencias
npm install

# Iniciar servidor de desarrollo
npm run dev
```

16.4 Puntos de Acceso (Desarrollo)

URL	Servicio
<code>http://localhost:5173</code>	Frontend (React)
<code>http://localhost:8000</code>	API Backend
<code>http://localhost:8000/docs</code>	Documentación API Swagger
<code>http://localhost:8000/health</code>	Health Check

16.5 Sembrar Base de Datos

```
# Via API (requiere backend corriendo)
curl -X POST http://localhost:8000/api/v1/admin/seed

# O via Python
PYTHONPATH=. python -c "from api.seed import seed_all; seed_all()"
```


17. Checklist de Produccion

- [] Cambiar `JWT_SECRET_KEY` a un valor aleatorio fuerte
 - [] Cambiar `DATABASE_URL` a PostgreSQL
 - [] Habilitar HTTPS en `nginx.conf` (descomentar bloque SSL)
 - [] Generar certificados SSL (`certs/cert.pem` , `certs/key.pem`)
 - [] Configurar `API_KEY` y `ADMIN_API_KEY` para proteccion de API
 - [] Configurar `CORS_ORIGINS` solo con dominios de produccion
 - [] Configurar `ANTHROPIC_API_KEY` para funciones de IA
 - [] Establecer `DEBUG=false` y `LOG_LEVEL=WARNING`
 - [] Revisar rate limiting en `nginx.conf`
 - [] Configurar montajes de volumen para datos persistentes
 - [] Configurar monitoreo y agregacion de logs
 - [] Ejecutar `docker-compose up -d --build`
-

18. Variables de Entorno

Variable	Valor por Defecto	Descripcion
DATABASE_URL	sqlite:///./ocp_maintenance.db	Cadena de conexion a la base de datos
SAP MOCK DIR	sap_mock/data	Ruta a datos mock SAP
JWT_SECRET_KEY	ocp-maintenance-secret-key-...	Secreto de firma JWT
JWT_ALGORITHM	HS256	Algoritmo JWT
JWT_ACCESS_TOKEN_EXPIRE_MINUTES	240	TTL del token de acceso
JWT_REFRESH_TOKEN_EXPIRE_DAYS	7	TTL del token de refresco
CORS_ORIGINS	http://localhost:5173,...	Origenes CORS permitidos
API_KEY	(vacío)	API key para proteccion de mutaciones
ADMIN_API_KEY	(vacío)	API key de administrador
ANTHROPIC_API_KEY	(vacío)	API key de Claude AI
GOOGLE_AI_API_KEY	(vacío)	API key de Google AI
DEBUG	false	Habilitar logging de depuracion
LOG_LEVEL	INFO	Nivel de logging Python
BACKEND_PORT	8000	Puerto backend Docker
FRONTEND_PORT	5173	Puerto frontend Docker
NGINX_PORT	8080	Puerto HTTP Nginx Docker
NGINX_SSL_PORT	8443	Puerto HTTPS Nginx Docker

Estadísticas del Proyecto

Metrica	Cantidad
Routers API	18
Endpoints API	100+
Modelos de Base de Datos	25+
Servicios Backend	23
Paginas React	23
Componentes UI	20+
Esquemas Pydantic	50+
Funciones Cliente API	50+
Idiomas Soportados	3 (EN, ES, AR)
Servicios Docker	3
Dependencias Python	27
Dependencias npm	30+

OCP Maintenance AI Platform — *Construido para la confiabilidad, diseñado para escalar.*